

A Robust and Flexible Operating System Compatibility Architecture

Takaya Saeki*

Department of Computer Science
The University of Tokyo
Tokyo, Japan
takaya.saeki@is.s.u-tokyo.ac.jp

Takahiro Shinagawa

Information Technology Center
The University of Tokyo
Tokyo, Japan
shina@ecc.u-tokyo.ac.jp

Yuichi Nishiwaki

Department of Computer Science
The University of Tokyo
Tokyo, Japan
nyuichi@is.s.u-tokyo.ac.jp

Shinichi Honiden

Faculty of Science and Engineering
Waseda University
Tokyo, Japan
honiden@nii.ac.jp

Abstract

Running applications for one operating system (OS) on another OS is useful in many cases. However, porting applications requires high development costs and running applications in a virtual machine poses resource sharing problems. A promising approach is to use an OS compatibility layer that converts the guest application's interface into the host one. Unfortunately, existing OS compatibility layers sacrifice either robustness or flexibility due to in-kernel subsystems or user-space-only implementations. This paper proposes a new architecture of OS compatibility layers that achieves robustness with almost user-level implementations while improving flexibility by exploiting standardized virtualization interfaces supported by most modern OSs. Our implementation of a Linux compatibility layer for macOS called Noah can run many of Ubuntu Linux binary applications, and a prototype implementation of a Linux compatibility layer for Windows confirmed the generality of our approach. Our experimental results demonstrated that the overhead of Linux kernel build time on Noah was 16%.

CCS Concepts • **Software and its engineering** → **Operating systems; Virtual machines; Software system structures; Software performance; Software reliability.**

*He is currently at Microsoft Development Co., Ltd.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

VEE '20, March 17, 2020, Lausanne, Switzerland

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-7554-2/20/03...\$15.00

<https://doi.org/10.1145/3381052.3381327>

Keywords Operating System Compatibility Architecture, Virtualization, Robustness, Flexibility

ACM Reference Format:

Takaya Saeki, Yuichi Nishiwaki, Takahiro Shinagawa, and Shinichi Honiden. 2020. A Robust and Flexible Operating System Compatibility Architecture. In *ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE '20)*, March 17, 2020, Lausanne, Switzerland. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3381052.3381327>

1 Introduction

Many applications are only compatible with a specific kind of operating system (OS). However, there are various cases where users want to run such applications on a different OS. For example, users may want to run a commercial desktop application for a specific OS that is different from the users' OS. Developers may want to use a build environment that consists of several toolchains and libraries of a specific OS (e.g., Linux) to build their applications on their computers that run a different OS (e.g., macOS). Given use cases like these, we need a casual way to run a part of other OS's applications in a seamless manner with the host applications.

One way to mitigate this problem is porting the applications. However, it requires considerable efforts by application developers and may also require significant efforts by users to set up runtime environments. The other way is to run the applications with the original OS in a virtual machine (VM) [43]. However, resource sharing between the guest and host applications is somewhat limited [48, 51], and some resource sharing is not supported because of the semantic gaps between two different OS kernels. While VM seamlessness has improved significantly in recent years, running two entire OS kernels is still resource consuming and could be excessive, especially on low-resource computers.

A promising approach is to use an OS compatibility layer. It can avoid the cost of porting applications by absorbing the difference between the guest and host interfaces at an

additional layer. It can also mitigate the resource sharing problems in VMs by allowing applications to run on the host OS natively; guest applications can communicate with host applications via host system calls. One way to implement OS compatibility layers is to exploit in-kernel subsystems [15, 36]. They can provide the flexibility to achieve binary compatibility because they can handle system calls and memory management events in kernel space. However, they lack robustness because a bug in the OS compatibility layers could lead to system crashes. For example, the former Windows Subsystem for Linux (WSL1) [36] had several bugs that could cause the blue screen of death of Windows [18, 26, 45].

An effective way to achieve robustness is to implement OS compatibility layers in user space because bugs in them do not affect the OS stability. Unfortunately, user-space-only implementations lack flexibility to implement all the functionalities for OS compatibility layers. For example, they cannot directly handle software interrupts for system calls without the support of the host OS kernels or patching application binaries. In addition, they cannot directly modify page tables to implement important memory management functions. As a result, user-space-only implementations can achieve only API compatibility, not full binary compatibility, and therefore applications must be modified or recompiled. Moreover, the inflexibility could lead to significant performance degradation. For example, Cygwin [8] is struggling with the `fork()` performance due to the lack of the copy-on-write capability [3, 9], and many developers refer to their poor build performance [34, 40]. MinGW [37] has given up full compatibility to gain performance. These observations indicate that robustness and flexibility are in a trade-off relationship in the traditional OS compatibility layers.

This paper presents a novel OS compatibility architecture that achieves both robustness and flexibility by exploiting virtualization technology. In this architecture, we launch a VM for each guest process and place the guest binary in the VM without the OS kernel. System calls issued by the guest process are trapped by a host process created for each guest process, called a *monitor process*. A monitor process emulates guest system calls by using host system calls and leveraging virtualization technology to trap software interrupts and manipulate page tables of the guest process. This architecture allows user-space implementations of OS compatibility layers with the support of the host OS functionality while achieving binary compatibility and reasonable performance with the support of virtualization technology.

We implemented a Linux compatibility layer for macOS, named *Noah*, based on the proposed architecture. Noah implemented many Linux subsystem emulations such as process management, memory management, virtual file systems, networks, and signals. Noah currently supports 172 out of 329 Linux system calls, and it can run many unmodified ELF binaries of x86-64 Linux 4.6 on macOS Sierra 10.12 or higher; we can build Linux kernels and run several X11

applications. Noah exploits Apple Hypervisor.framework (AHF) [2], the standard virtualization interface supported by macOS, and therefore we do not need to modify the macOS kernel. Our experimental results demonstrated that the overhead of Linux kernel build time on Noah was 16%.

We also implemented a prototype Linux compatibility layer for Windows, named NoahW, to show the generality of our approach. Although this implementation is still in an early stage, we confirmed that simple Linux binary applications run on Windows with NoahW. NoahW exploits Intel Hardware Accelerated Execution Manager (Intel HAXM) [24] as the virtualization interface, and therefore we did not need to modify the Windows kernel. Our experimental results demonstrated that repeating `fork()` and `wait()` in a process with 1 GiB data was 4.38 times faster on NoahW than Cygwin.

Our previous workshop paper [41] showed the basic idea of this OS compatibility architecture. However, it gave only a brief description of the design and implementation on the OS compatibility architecture, and its evaluation was preliminary. In this paper, we present a more detailed description of the design and implementation of the proposed architecture. We also fixed several bugs, insufficient functionalities, and performance problems in the previous Noah implementation to withstand detailed performance evaluation. Based on this improved implementation, we show the results of comprehensive performance evaluations using a primitive benchmark, micro benchmarks (Imbench), and macro benchmarks (Phoronix Test Suite) that reveal the advantages and disadvantages of our approach. We also newly implemented a prototype of NoahW to show the generality of our approach and demonstrated that the copy-on-write performance was significantly enhanced in NoahW compared to Cygwin.

Our research contributions are as follows:

- We propose a new OS compatibility architecture that achieves both robustness and flexibility by exploiting virtualization, and discuss the advantages of our architecture compared to other architectures.
- We present a practical implementation of the proposed architecture that can run many Linux binaries on macOS, and a prototype implementation that can run simple Linux binaries on Windows.
- We evaluate our implementations from the viewpoint of their characteristics and performance and demonstrate its effectiveness.

The remainder of this paper is organized as follows. Section 2 shows related work. Section 3 explains the design and Section 4 describes the implementation of our Linux compatibility layers. Section 5 presents evaluations and Section 6 concludes this paper.

2 Related Work

2.1 OS Compatibility Layers

Many OS Compatibility layers can run application binaries of other OSs without modifications with the support of in-kernel subsystems. For example, Linux binaries run on NetBSD [11], FreeBSD [15], and Windows [36]; IRIX binaries run on NetBSD [12]; Mac OS binaries run on NetBSD [13] and Linux [19]; iOS binaries run on Android [1]. The advantage of the in-kernel subsystems is the flexibility to achieve compatibility and performance, i.e., the ability to intercept system calls without binary modifications and to implement the copy-on-write capability. However, robustness is sacrificed because a bug could cause kernel crashes.

Several user-space-only implementations of OS compatibility layers are also available. For example, Cygwin [8] and MinGW [37] can run UNIX-like applications in the Portable Executable (PE) binary format of Windows. The main advantage of user-space implementations is robustness by avoiding kernel modifications. Another advantage is the development cost; user-level development is in general easier than kernel-level development, and user-space implementations can exploit various cross-platform software libraries like C++ Boost and high-level languages like Rust and Go. The availability of cross-platform software could also contribute to improve the reusability of OS compatibility layers. However, existing user-space-only implementations lack the flexibility to achieve full binary compatibility or sufficient fork() performance for development environments.

Wine [53] achieves the binary compatibility of Windows applications with only user-space libraries. This is because Windows changes the system call numbers among versions and requires applications not to issue system calls directly. Wine also does not need to implement the copy-on-write capability because Windows does not support it. However, the compatibility can be realized largely due to the unique architecture of Windows. In general, our approach has the advantage of achieving binary compatibility and the copy-on-write capability of UNIX-like guest application binaries with user-space-only implementations.

Binary translation contributes to improvement of binary compatibility in user space. For example, Foreign LINUX (flinux) [44] performs binary translation to replace system call instructions with calls to the OS compatibility layer. It also mitigates segment register handling and redzone calling convention on the stack. However, memory layout configuration is not as flexible because a guest process shares memory space with the corresponding host process. Wabi [21] and FX!32 [7] exploit binary translation to perform CPU emulation. However, CPU emulation significantly degrades performance. Although our approach does not support CPUs different from the host architecture, it achieves native execution of guest binaries and good performance with the support of the copy-on-write capability.

2.2 VMMs

VMMs can run applications of the other OS in a VM along with the OS itself as the guest. The main advantage of using VMMs is the full compatibility; applications including their runtime environments will run without modification. The performance is also reasonable due to recent advances in virtualization technology. However, resource sharing between the VM and host OS is not necessarily seamless because there exist two different OS kernels; they manage resources independently based on the policy of each OS.

Much effort has been devoted to improve the seamlessness between the guest and host OSs. File sharing and copy-and-paste between them have been supported early on [49]. The VMware Fusion Unity View can mix application windows of Windows and macOS [50]. The Windows Subsystem for Linux 2 [29] further integrates the guest Linux into the host Windows; the OS boots in a few seconds [29], files can be seamlessly shared via the 9p protocols [30], and memory is efficiently shared by Memory Reclamation [31]. Therefore, VM seamlessness might be sufficient in some use cases.

However, there is still room to improve seamlessness by integrating applications at the process level. For example, inter-process communications (IPC) such as pipelines, sockets, and signals between the guest and host applications could be useful for daily command-line use in application developments. The communication between the guest and host application processes could become more efficient if they are scheduled by a single OS kernel. File access could become faster if the guest applications can access files directly with the native host system calls. Moreover, using VMs consumes resources to some extent for managing two OS kernels. Therefore, running only application processes without the guest OS kernel could be more lightweight and useful on low-resource computers like laptop computers.

2.3 Similar Architectures

Microkernels could achieve both robustness and flexibility of OS compatibility layers with user-space handling of system calls and memory management. For instance, Mach supports 4.3BSD binaries [17] and L4 supports Linux binaries [20] with user-space servers. However, microkernel interfaces are usually very primitive and the development cost of OS compatibility layers could become high. Moreover, modern popular commercial OSs such as Windows and macOS do not allow external developers to exploit microkernel features.

Xax [10] can run some desktop applications in web browsers. It proposed the concept of picoprocess that is an isolated native-code execution environment with a narrow system-call interface. Embassies [22] runs applications from multiple vendors on the client based on the picoprocess concept. Another work [23] runs rich POSIX applications with the picoprocess interface. Drawbridge [39] runs Windows 7 applications with a library OS on a platform adaptation layer

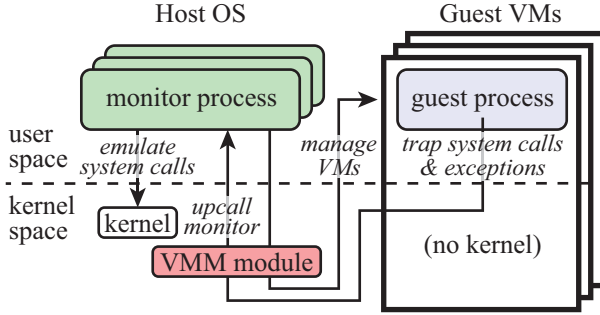


Figure 1. Design of proposed OS compatibility layer.

and security monitor in host Windows. Graphene [46] is a Linux-compatible library OS executed within a picoprocess. All of these systems run guest applications on a execution environment with a narrow system-call interface for the isolation purpose, while our approach use normal system call interfaces without aiming for strong isolation.

Dune [4] resembles our work in that guest programs run in VMs with higher-level interfaces than machine architectures. However, the goals of Dune are different; it aims to provide user programs direct access to hardware features, whereas we emulate the kernel interface of a different OS. We use only ring 3 in the VMX non-root mode of Intel VT-x for running guest processes, whereas Dune runs processes in different rings and VMX modes. Multiverse [28] has a similar goal as that of Dune. Multiverse provides Linux applications the ability to use the privileged Hybrid Runtime (HRT) environment. It leverages a VMM to trap privileged operations and emulate Linux behaviors in HRT. However, Multiverse has no host OS under a VMM, and it emulates privileged operations by using the actual Linux kernel that is running cooperatively with the HRT. By contrast, our architecture handles privileged operations in the host OS to realize resource sharing and seamless communication.

Socket outsourcing [14] redirects network access from the guest OS to the host OS at the socket layer to improve network performance. Socket outsourcing is similar to our approach in that guest resource access is forwarded at a higher-level interface. However, socket outsourcing requires modifications to both guest and host OS kernels.

3 Design

Figure 1 shows the design of the proposed OS compatibility layer. It consists of three components: guest VMs, VMM module, and monitor processes. A guest VM is a container of a guest binary, and a guest process runs inside a guest VM. The VMM module is a standard component running in the host OS kernel space, and it provides VM management interfaces to the user-space programs. A monitor process is a regular process of the host OS, and it manages a guest VM through the VMM module.

A guest application is executed in the following way. First, a monitor process is launched. It creates a guest VM through the VMM module and loads the guest binary image into the memory space of the VM. Then, the monitor process requests the VMM module to start execution of the VM from the entry point of the guest binary, thereby passing control to the guest process. While the guest process is running, it issues a system call. The system call is trapped by the VMM module, and the VMM module up-calls the monitor process. The monitor process emulates the trapped guest system call by using system calls of the host OS. The monitor process then returns the result of the emulated system call to the VM via the VMM module, and control is returned to the guest process. When an exception occurs inside the VM, it is handled by the monitor process. The monitor process may emulate the copy-on-write capability or send signals with the support of the VMM module.

3.1 Guest VMs

A guest VM is an instance of the guest execution environment. It is created for each guest process and provides a virtual address space for the guest process. The virtual address space is configured by a page table, which is set up and manipulated by a monitor process through the VMM module. The guest binary image is loaded onto the virtual address space and the binary is executed when a virtual CPU is assigned to this VM.

We assume that guest VMs are implemented using a hardware-assisted virtualization technology. This means that most CPU instructions are executed directly in the VMs without software intervention, while sensitive instructions are trapped by the CPU. We assume that the CPU can trap system call instructions, page faults, and other exceptions. This is important for achieving binary compatibility of guest applications. In Figure 1, system calls and exceptions generated in a guest VM are trapped by the VMM module, and control is passed to the monitor process via upcall.

Because all exceptional events that occur in the user space of a guest VM are handled by the monitor process, we place no kernel image in the VM, as shown in Figure 1. However, we place a few data structures for the management purpose, and we may place a trampoline code to trap system calls and exceptions if the implementation requires it. The details are discussed in Section 4.

3.2 VMM module

The VMM module is a small driver of the CPU’s virtualization functions. The VMM module provides several interfaces to manage VMs. For example, it provides APIs to create and destroy a VM, create and destroy a virtual CPU, read and write VM states, modify the page table of a VM, and pass control to a VM. Especially, it is important to provide interfaces to obtain control when the program inside a VM issues a system call or causes an exception for achieving

binary compatibility. The VMM module runs inside the host kernel space and issues privileged instructions of CPU virtualization functions.

Because the VMM module runs inside the kernel space, any bugs in this module could compromise robustness. However, we believe that it is relatively easy to keep the VMM module robust for the following reasons. First, we assume that it is a small driver for virtualization functions such as Intel VT-x and AMD SVM. Small software tends to have fewer bugs [16], and therefore, it is easy to make such software robust. Second, it does not depend heavily on the host OS kernel. Therefore, it is unlikely to generate bugs owing to complex interactions with OS kernels. Third, such modules are already integrated into many modern OS kernels, such as AHF of macOS, Windows Hypervisor Platform of Windows [35], KVM of Linux, and vmmapi of FreeBSD. Therefore, it will be supported and maintained as well as the OS kernel. Finally, it is also used for purposes other than OS compatibility, such as creating normal VMs. Therefore, we can expect that their implementations would soon become mature and stable compared to other in-kernel subsystems of OS compatibility layers.

3.3 Monitor Process

The monitor process is the main component of the proposed OS compatibility layer. It provides OS compatibility with the support of the host OS and the VMM module. It performs several tasks. First, it prepares an execution environment of the guest application. It creates a VM and sets up the page table via the VMM module, loads the application binary with its own program loader, and passes control to the entry point of the binary program. Second, it handles system calls and exceptions that occur during execution of the guest binary. It emulates the guest system calls with the host system calls, handles page faults by managing the page table, sends signals upon executing an undefined instruction or coming across a floating-point exception, and so on. Finally, when the guest process exits, it frees allocated resources, destroys the corresponding VM, and terminates itself.

Our monitor process design has several advantages in terms of robustness, flexibility, development cost, and seamlessness. We describe each of them below.

Robustness A monitor process is a user-space application on the host OS. Therefore, bugs in a monitor process will not affect other guest processes, host processes, or the entire system. Even if a monitor process terminates unexpectedly owing to a bug, only the corresponding guest process is affected, and the resources allocated by the monitor process are safely freed by the kernel and the VMM module. This design contributes to improving the robustness of the OS compatibility layer.

Note that `setuid` programs can pose problems related to robustness. If the `setuid` bit is set in a guest program, such as

in `sudo`, and we want to run the program with administrator privileges, the corresponding monitor process must have administrator privileges as well. This requires that the `setuid` bit of the program file of the monitor process must be also set. In this case, if there is a bug in the monitor process, the stability of the overall system could be compromised. However, this issue can be mitigated by quickly dropping unnecessary privileges if the guest program is not a `setuid` program. Although the `setuid` semantic is known to be complex, there is a good guideline for properly handling it [5]. In addition, we can choose not to support `setuid` programs. It is a trade-off between usability and robustness.

Flexibility Flexibility is divided into two properties: compatibility and performance. As for compatibility, our design can achieve binary compatibility without need for modification of guest programs or host kernels. This is possible because the monitor process can intercept system calls and exceptions with the support of the VMM module, even though the monitor process runs in the user space. This is the powerful ability of the monitor process to achieve binary compatibility of the guest programs.

As for performance, the copy-on-write capability is the most important one to achieve from the viewpoint of the `fork()` performance in UNIX-like guest applications. Our design can achieve the copy-on-write capability because the monitor process can manage the page table of the guest VM via the VMM module. When the guest process issues a `fork()` system call, the monitor process, too, issues a `fork()` system call (or another system call to create a process). The two monitor processes have a shared memory and map it to both the parent and the child guest VMs with the read-only permission. When one of the guest processes write to the shared memory region, a page fault occurs, and the corresponding monitor process is up-called. The monitor process first copies the memory contents and then gives write permission to the memory region.

If an OS compatibility layer does not support the copy-on-write capability, it must copy the entire memory region of the process when the process issues a `fork()` system call. This leads to a significant cost of memory copy, especially in the case when the process memory is large. The cost could be a waste, especially when the process immediately issue a `exec()` system call. Therefore, supporting the copy-on-write capability significantly impacts the performance of OS compatibility layers.

Development Cost As in other user-space OS compatibility layers, our design can exploit the functionalities of the host OS. If the guest and host OSs have similarities, most system calls issued by the guest process can be simply forwarded to the host OS. Most OSs have similar functionalities such as process management, memory management, file systems, and networking. Therefore, it is relatively easy to convert guest system calls into host system calls

compared to implementing system calls from scratch or with low-level APIs such as in microkernels. In addition, system calls are a relatively well-defined interface and do not change frequently over time. This property opens the way to reuse the implementation of an OS compatibility layer for another OS compatibility layer. In addition, the proposed design allows the developer to use any cross-platform utility libraries or high-level languages.

Seamlessness Similar to other OS compatibility layers, one advantage of the proposed design is seamlessness. Because a monitor process is created for each guest VM, its resource management is integrated into that of the host OS. For example, the scheduling of guest processes is handled in the same way as in the host processes. This allows for fair scheduling among multiple guest and host processes. In addition, it can help avoid synchronization problems in ordinary VMs, such as the lock holder preemption [47]. Moreover, file systems become a shared resource: the guest and host OS can share the same file system tree. IPCs, such as pipes, sockets, semaphores, and shared memory, between guest and host processes are possible in the same way as if they were running on the same OS.

4 Implementation

We implemented Linux compatibility layers for macOS (Noah) and Windows (NoahW) based on the proposed design. Both implementations target x86-64 Linux 4.6. Noah runs on macOS 10.12 Sierra or higher and uses AHF as the VMM module, which is a built-in library of macOS. NoahW basically runs on Windows 7 and higher, but it has been tested only on Windows 10, and it uses Intel HAXM, a library developed by Intel for 5 years, as the VMM module. We currently support only Intel CPUs and do not support AMD CPUs because the VMM module is CPU dependent.

4.1 Boot Process

A guest Linux process is created when the `noah/noahw` command is executed or a guest process issues a `fork()`. When a guest process is being created, the monitor process first launches a new VM by using the VMM module. Then, to avoid injecting the custom boot code into the VM, the monitor process manipulates the VM registers so that the VM directly enters the x86-64 long mode. In Noah, the monitor process additionally configures the VM such that the VMM module traps all exceptions inside the VM and up-calls the monitor process. This is achieved by setting the *exception bitmap* of Intel VT. Additionally, to trap system calls, the monitor process leaves the `IA32_EFER.SCE` bit cleared, which means that the `SYSCALL` instruction is trapped as the “undefined instruction” exception. Moreover, to prevent any code from running in privileged mode in the VM accidentally due to bugs, the monitor process initializes some control

registers (such as `CR0` and `CR4`) and model-specific registers (including `IA32_EFER`) with empty settings.

In NoahW, we cannot directly cause VM exits on system calls and exceptions, because the current HAXM does not support APIs to configure the exception bitmap. Therefore, we inject trampoline code as exception handlers in the VM, although this involves extra costs of context switches.

In x86-64, a few system registers need to hold physical memory addresses of memory data structures (e.g., page tables and segment descriptor tables). To allocate such data structures, we reserved a region of 1 GiB in the physical address space in the VM, which is not mapped in the user space. The monitor process allocates data structures from this region. (See Section 4.3 for the detail).

4.2 ELF Loader

We implemented an original ELF loader to load Linux ELF executable files into VMs. The loader is invoked after the `noah/noahw` command is executed or a guest Linux process issues the `execve()` system call. When the loader is passed a path to a Linux ELF executable file, it first opens the Linux ELF loader file (`ld.so`) through the virtual file system in the monitor process. It then uses the internal version of `mmap()` to map the loader file into the guest Linux address space in the VM. After setting up the execution environment of `ld.so`, it passes control to `ld.so` with the original executable file as an argument. Finally, `ld.so` loads the executable file, constructs memory segments, and resolves dynamic linked libraries in the emulated environment as usual.

In Noah, the ELF loader supports the `setuid` bit as well. If the monitor process is executed as root and the `setuid` bit of the target executable file is set, the monitor process changes the effective user ID of the guest Linux process to that of the file owner. For example, a `setuid-root` Linux command like `sudo` can write to files owned by root. Note that guest Linux processes share the same user name and ID with the host macOS.

4.3 Memory Management

Because we use a VM to run a guest process, we need to manage page tables by ourselves. In general, a VM involves two page tables: the VM’s page table and the VMM’s nested page table called the extended page table (EPT) in Intel CPUs. To avoid handling two page tables, we should fix one page table and manipulate only the other. Which one to choose is a design choice. We chose to fix the VM’s page table and manipulate the EPT for easy debugging. In the early stage of development, we designed a VM and its monitor process to share the same mapping from virtual addresses to physical addresses for a specific region. To do the same in the VM’s page table, we must obtain the physical address corresponding to the virtual address. Unfortunately, macOS does not support such APIs. By contrast, AHF supports an

API to set the physical address of an EPT entry by specifying a virtual address of the monitor process.

In the VM's page table, we use straight mapping where the virtual address is identical to the physical address. This mapping is simple but is limited by the hardware bus width. The physical address width of the most current Intel CPU series is no more than 39 bits, whereas the virtual address width is 48 bits. Therefore, we cannot map the upper 9-bit virtual address space in a VM with this mapping. Fortunately, the only case we observed in which the space was used in the current Linux was the default start address of the user stack. To manage this, we change the start address to the highest address in the available address bus. This is not a problem because Linux applications usually do not expect the stack address to be specific. In addition, we have never observed any application exhausting 256 GiB (i.e., the maximum addressable size with 39 bit) of the virtual address space. The exception of straight mapping is the top 1-GiB region. No physical page is mapped here in the virtual address space to hide it from the user space. It is used to locate system data structures such as page tables and segment descriptor tables (described above in Section 4.2). Consequently, the VM's page table has the 511-GiB straight mapping and 1-GiB empty mapping.

Implementation of the other part of memory management is rather simple. We manage memory regions with the `mm_region` structure, which is similar to the `vm_area_struct` of the Linux kernel. When a guest Linux process issues `mmap()` or other memory-related system calls, the monitor process manipulates EPT by using the VMM module. Because the API of the VMM module to manage EPTs accepts virtual addresses of the monitor process, the corresponding physical pages are managed by the VMM module and the host OS.

To support multiple guest processes, monitor processes need to communicate with each other. We selected to use shared memory, mainly for performance reasons. When the `noah/noahw` command starts, it pre-allocates a few gigabytes of the shared memory region, called the central shared memory. Data structures to be shared among monitor processes are allocated from this region. Note that the central shared memory consumes little physical memory owing to lazy page allocation.

4.4 Process Management

Noah and NoahW manages process states with the process structure that includes the `mm_region` structures, `futex` structures, signal handlers, and `vfs` structures. They are stored in the central shared memory area, described in Section 4.3. Although most aspects of process management, such as scheduling, can be left to the host OS, Noah and NoahW need to perform important tasks related to `fork()` and thread management. We describe each of them in detail below.

4.4.1 Fork and thread creation on macOS

We implemented a subset of `clone()`; only simple forking and thread creation are supported. Unfortunately, AHF does not support the `fork()` of a process holding a VM. Therefore, we perform the following operations. First, the monitor process saves the current VM state and destroys the VM. Second, the monitor process forks. Third, each process launches a new VM. Finally, they restore the saved state and start execution. We synchronize VM restarts by using condition variables to avoid race condition. As for threads, we create one thread of the monitor process for each guest thread.

4.4.2 Fork and Copy-On-Write on Windows

The `fork()` implementation in Windows is complex because Windows does not support `fork()`. NoahW realizes `fork()` and copy-on-write by using shared memory and virtualization. First, the monitor process allocates a shared memory region for each guest Linux memory region in Windows with `CreateFileMapping()`, which we call a host-shm. This host-shm can be shared among monitor processes that manage guest processes in a parent-child relationship. Note that host-shms are different from the central shared memory mentioned in Section 4.3.

Then, the monitor process emulates `fork()` as follows: (1) The monitor process allocates a process structure in the central shared memory for the new guest child process and copies data from the guest parent process. This data structure includes the `mm_region` structures. (2) The monitor process remaps the corresponding guest memory region of the guest parent process as read-only in the VM. (3) The monitor process saves the current VM states and creates a new monitor process. (4) The new monitor process creates a new VM and maps the host-shms into the new VM's address space as read-only mappings based on the `mm_region` structures. (5) The new monitor process restores the VM states and starts execution of the guest child process.

Copy-on-write is handled in the following way: When a process requests write access to a read-only region, the VMM traps the access and notifies the corresponding monitor process. If the page is mapped by multiple guest processes, the monitor process allocates a new host-shm and copies the contents of the page that is write-accessed. It then maps the new host-shm to the written address of the guest VM as writable mapping. If the page is not mapped by any other processes, the monitor process simply remaps the page as writable mapping in the VM.

The essential steps above are basically simple. However, their concrete implementation is more complex. One memory region of a guest process could correspond to multiple host-shms because copy-on-write could introduce a new host-shm, as described. Therefore, each `mm_region` structure also needs to manage multiple host-shms mapped

in its region. This is managed by its structure member called `host_fmappings`. In addition, copy-on-write requires maintaining reference counters of pages. For this purpose, we maintain an address-range-based reference counter. It remembers the number of references from the `mm_region` structures that each page of a host-shm has. The `host_fmappings` structure and this reference counter are implemented with red-black trees, a key of which is a non-overlapping address range.

4.5 Virtual File System

In Noah, we implemented a virtual file system. File access from guest Linux processes is basically forwarded to the macOS file system. However, to support Linux-style file system trees, we emulate the virtual file system (VFS) of Linux in the monitor process. The VFS consists of a path translator and an object-oriented programming (OOP) component. The path translator converts a virtual path to the host path by resolving symbolic links and virtual mount points. The OOP component provides an interface to install custom file systems. The VFS allows us to expose macOS's root file system to Linux programs without breaking the Linux user space. In addition, virtual file systems such as `sysfs` and `procfs` can be implemented upon this facility. The implementation of the VFS is designed to be independent from the host OS architecture. Therefore, it can be easily ported to NoahW, which is our future work.

4.6 Other Subsystems

In Noah, most other Linux system calls are passed-through to the host OS with flag conversion and structure adjustment. For example, the monitor process handles `getpid()` by simply calling the equivalent system call of the host OS and returning the value to the guest process. Most signals sent to a monitor process are routed to the corresponding VM with proper conversion. Signal handlers of the monitor process simply record the arrivals of the signals. Before the next entrance to the VM, the monitor process checks the records, and if signal arrivals are recorded, it creates a new signal stack frame in the VM and sets the IP to the registered signal handler. Supporting these features in NoahW is future work.

4.7 Discussion

Based on the experiences of implementing Noah and NoahW, we summarize our insight into the characteristics of the VMM module that are suitable for implementing the proposed architecture.

Configurable VM exit strategy The VMM module should support a mechanism to trigger a VM exit event when the guest process issues a system call. As described in Section 4.1, the mechanism to trap system calls from the guest process differs between Noah and NoahW. In Noah, since AHF allows us to configure the Intel VT-x exception bitmap, a system

call from the guest process causes a VM exit event and Noah can trap it immediately. On the contrary, HAXM does not expose an API to access the exception bitmap or the other way to configure when to trigger VM exits. Thus, we had to put a trampoline system call handler just to trigger VM exit in a VM's kernel memory space. Therefore, in Intel VT-x, it is desirable to expose API to access the exception bitmap in the VMM module. For other virtualization mechanisms, such as AMD SVM, different APIs might be necessary.

No unnecessary resource limitations The VMM module should not impose more resource limitations than the actual processor has. Unfortunately, some VMM modules have additional limitations on the number of VMs and the number of memory mappings which Intel VT-x does not have. For example, KVM has the limitation on the number of memory mappings, `KVM_CAP_NR_MEMSLOTS`, which is usually 10 [27]. This is probably because KVM abstracts a memory mapping between the guest and host as "a memory slot." It would not cause any problem to implement typical virtual machine monitors. However, our architecture creates a relatively small memory mapping every time a guest process issues a memory-related system call such as `mmap()`. Therefore, it is desirable not to place unnecessarily small limits on the number of memory mappings. Similarly, HAXM limits the maximum number of VMs to 8 [25]. Because our architecture uses one VM per one guest process, it is a serious limitation on the number of processes that can run simultaneously. Due to this limitation, we need to modify HAXM to develop NoahW. AHF has no such limitations.

Performance The performance of VM creation, VM entry, VM exit, and reading/writing of the VM registers significantly affects the performance of our architecture. Every time a guest process issues a system call, VM exit and entry occur, and a monitor process needs to read registers. VM creation is necessary for each `fork()` system call. As we will show in Section 5, the performance difference between AHF and HAXM significantly affect the basic system call performance of Noah and NoahW. Therefore, the performance of these primitive virtualization functions should be highly optimized.

5 Evaluation

This section presents evaluation of the characteristic and performance of our architecture. As for the characteristic, we evaluated our architecture from the following viewpoints: robustness, flexibility, development cost, and seamlessness. As for performance, we ran a couple of benchmarks and compared it with the corresponding native OSs.

We evaluated Noah on a MacBook Pro (13-inch, 2017) with a two-core / four-thread 3.5 GHz Intel Core i7-7567U CPU, 16 GB 2133 MHz LPDDR3 memory, and 500 GB SSD. We ran the user-space binaries of Ubuntu 16.04 on macOS High

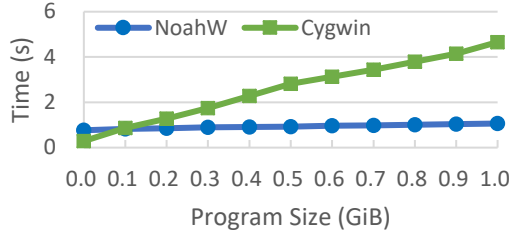


Figure 2. Copy-on-write performance in Windows 10

Sierra 10.13.3. For NoahW, we performed the evaluations on a Surface Pro 2017 with a two-core / four-thread 2.50 GHz Intel Core i7-7660U CPU, 16GB DDR3 memory, and 512 GB SSD. The host OS was Windows 10 Professional Version 1709. The version of HAXM was 6.2.1.

5.1 Characteristic

Robustness During the development process, Noah crashed many times. However, these crashes hardly caused a macOS or Windows kernel crash, because Noah is a user-space process, and a bug in Noah leads to a successful process termination which are correctly handled by the kernel. The only kernel crashes we faced were related to AHF. When we tried various configurations of VMs to start the Linux process in the 64 bit mode, the kernel crashed a few times owing to an AHF bug.

As described in Section 3.2, VMM bugs will affect the robustness in our architecture. However, we only faced kernel crashes in the early stage of the development, and they did not occur after the VM configuration was established. Considering the number of bugs (actually two bugs) that occur in the development of OS compatibility layers, the number of kernel crashes is significantly low, and therefore, robustness of the proposed architecture is relatively high.

Flexibility We confirmed that our implementations achieved binary compatibility with Linux applications without the support of in-kernel subsystems. Especially, Noah could run many real-world Linux binaries, including package managers such as apt and pacman; development tools such as gcc, vim, make, and Ruby; daily command-line tools such as bash and ls; network tools such as nc; and X applications such as xeyes, xfwrite, and DooM3. In addition, we were able to finish lmbench [32].

In addition, we confirmed that NoahW successfully implemented the copy-on-write capability. We ran a benchmark program that repeated fork() and wait() 10 times. We changed the size of a global array from 100 MiB to 1 GiB and measured the execution time. We compiled the benchmark program in Linux and ran it on NoahW. For comparison, we compiled and ran the program in the Cygwin environment. As shown in Figure 2, the fork() performance did not degrade in NoahW, while it degraded significantly in Cygwin in proportion to

the size of the global array, as Cygwin copied the entire global array from the parent to child process on fork().

Development Cost We counted the source lines of code (SLOC) of Noah with sloccount [52]. The total SLOC was 12,706, of which ‘src/’ had 8,328 lines, ‘include/’ had 4,073 lines, and ‘lib/’ had 305 lines. The development effort was estimated as 2.89 person months. The ‘include/’ directory contained long definitions of simple conversion tables for system calls, various constants, and so on. Therefore, the effective SLOC will be slightly lower.

Compared to VMMs, which have hundreds of thousands of lines of code, and Linuxulator, which runs inside the FreeBSD kernel (31,886 SLOC for linux/, linuxkpi, linprocfs, linsysfs), we can achieve binary compatibility efficiently with fewer lines of code, and therefore, require less development cost by exploiting the functionality of the host macOS.

Seamlessness We confirmed the seamlessness of the file system. Noah mounts ‘\$HOME/.noah/tree’ of macOS on ‘/’ and \$HOME on \$HOME. All files are stored in the native file system of macOS. Therefore, the full file system tree of the Linux environment can be accessed from macOS.

In addition, we confirmed that processes on macOS and Noah could communicate with each other via System V shared memory. In this experiment, a process on macOS created a shared memory area by using shmget() with a key, attached it to the address space, and wrote a value to the memory area. A process on Noah obtained the id of the shared memory with shmget(), attached it to the address space, and read the written value on the memory area. This cross-OS communication shows the seamlessness of our architecture.

5.2 Performance

For performance testing, we ran a couple of benchmarks on Noah and NoahW. The benchmarks are divided into three parts: primitive, micro, and macro benchmarks. The primitive benchmark shows the basic system call performance and its breakdown. The micro benchmark shows the performance of system calls measured with lmbench. The macro benchmark shows the performance of applications. Since NoahW is premature, we only used Noah for micro and macro benchmarks.

5.2.1 Primitive Benchmark

To show the primitive system call performance, we measured CPU cycles to call the dup() system call. We chose dup() because it could not be batched or cached [6]. We found that dup() took 2,770 cycles in macOS and 10,538 cycles in total in Noah, respectively. By contrast, dup() took 2,118 cycles in Windows and 37,464 cycles in total in NoahW, respectively.

Figure 3 shows the breakdown of dup() on Noah and NoahW. We used VMXbench [42] to measure the VM exit/enter costs and recorded the cycle counter in Noah and NoahW in each stage to obtain the other cycle numbers.

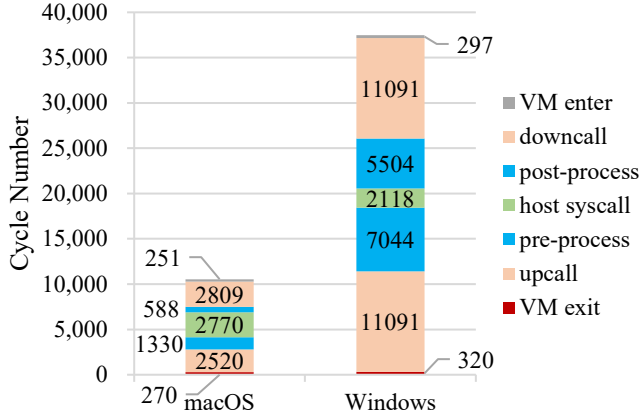


Figure 3. dup() breakdown

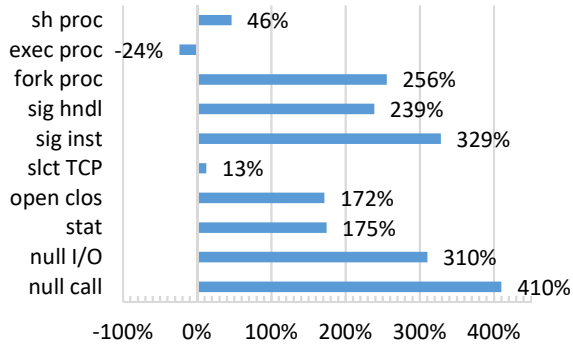


Figure 4. lmbench (Processor, Processes)

We could not divide the cycles of upcall and downcall in HAXM, so we used the equally divided value (11091 cycles). As shown in the figure, the VM exit/enter cost (521 or 617 cycles) was not very high. However, upcall and downcall by the VMM module required many cycles (5,213 cycles for AHF and 22,182 cycles for HAXM). Regarding the high cost of pre-process and post-process in NoahW (22182 cycles in total), we observed that most of the cost was to access registers via HAXM. This is caused by the HAXM's API that reads and writes all registers of a processor even when we want to read or write only one register. Using Windows Hypervisor Platform would reduce this cost significantly. On the other hand, AHF has an API to read one register.

5.2.2 Micro Benchmark

To measure the overall system call performance, we ran lmbench-3.0-a9 [33]. We only ran the “OS” benchmarks, which stresses the OS by issuing many system calls. All the results are shown as the overhead (%) of Noah compared to native macOS.

Figure 4 shows the result of “Processor, Processes.” We found that “null call” and “null I/O” were very slow (overheads were 410% and 310%, respectively) because these

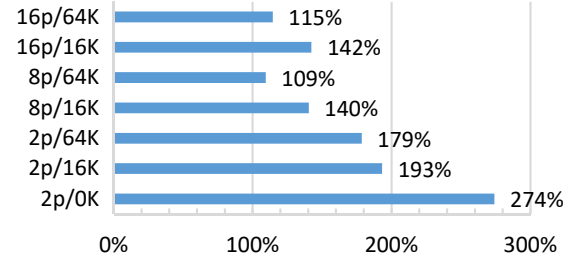


Figure 5. lmbench (Context switching)

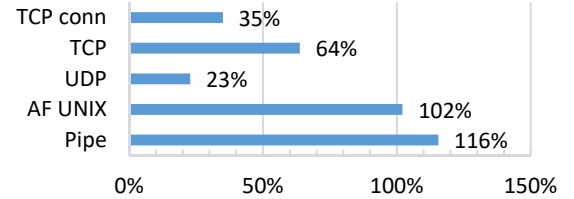


Figure 6. lmbench (Local Communication latencies)

benchmarks were significantly affected by the basic system call performance. Similarly, simple system calls such as “stat,” “open close,” and signal handling were slower. By contrast, the overheads of “select TCP” and “shell process” were relatively low owing to the long processing time of the corresponding macOS system call.

One notable result was the performance of exec(). Noah implements exec() by itself and does not call the corresponding system call of macOS. In addition, it implements its own ELF loader, and maps the executable file with the support of the VMM module. It is a relatively simple operation and took 1,485μs, while macOS took 1,952μs which was somewhat slower, probably owing to the internal structure derived from the microkernel.

Figure 5 shows the result of “Context switching.” We performed this experiment in a single-core environment to obtain accurate results. Noah incurred 109-274% overhead. Especially, the “2p/0K” case incurred the highest overhead, which is reasonable considering that the basic system call overhead is incurred in addition to the context switch cost of the monitor processes.

Figure 6 shows the result of “Local Communication latencies.” The overheads were in the range of 23-116%. Unlike the context switching experiments, we measured the latencies on a normal multi-core environment. Therefore, two communicating processes would run on each core, reducing the cost of context switching.

Figure 7 shows the result of “File & VM system latencies.” The result shows that Noah incurred a 4-42% overhead on basic file-related system calls. Notable results are those of “Page Fault” and “Prot Fault.” These results related to page table handling, which is implemented by Noah, instead of

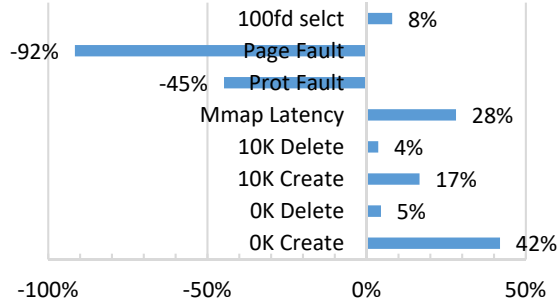


Figure 7. Imbench (File & VM system latencies)

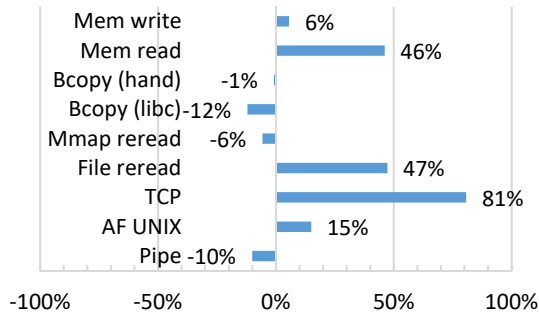


Figure 8. Imbench (Local communication bandwidths)

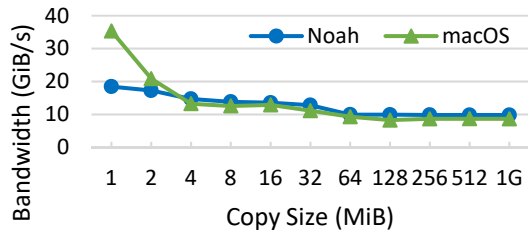


Figure 9. Imbench bcopy bandwidth against copy size

the native macOS kernel. For the same reason as `exec()`, Noah was considerably faster.

Figure 8 shows the result of “Local Communication bandwidths.” Basically, the performance in Noah is inferior to that in macOS, while a few of them (“Pipe,” “Mmap reread,” “Bcopy (libc),” and “Bcopy (hand)”) performed better. These results can possibly be ascribed to the difference in page fault handling speed. Taking “Bcopy” as an example, as the copy size increased, the number of page faults increased owing to on-demand paging. The page faults were handled by the macOS kernel when executed on macOS, while they are handled by Noah when executed on Noah. As shown in Figure 7, the page fault handling of Noah was considerably faster than that in macOS. Therefore, memory copy speed in Noah was faster than that in macOS. To justify this, we plotted the detailed results of Imbench bcopy, as in Figure 9. As the buffer size increased, the memory copy bandwidth

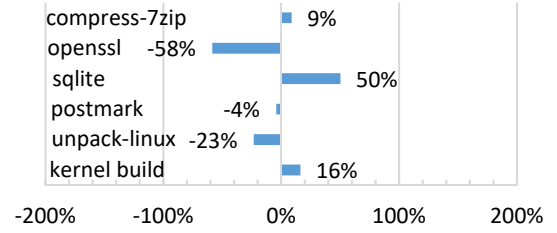


Figure 10. Macro Benchmark

in macOS degraded. Therefore, applications on Noah will show better performance when they access a large memory region.

5.2.3 Macro Benchmark

To show the performance in real applications, we used Phoronix Test Suite v7.6.0 [38] and measured the performance of “compress-7zip,” “openssl,” “sqlite,” “postmark,” and “unpack-linux.” In addition, we measured the kernel build time of Linux 3.4.113. In macOS, we manually modified the kernel so that its minimal configuration could be compiled with cross tool chains. The configuration was “allnoconfig” and we built “vmlinux.”

Figure 10 shows the results. We found that Noah was sometimes faster and sometimes slower. “compress-7zip” incurred only 9% overhead, while “openssl” was 58% faster. We confirmed that “openssl” hardly issued any system calls or accessed large memory regions in both environments during the benchmark, so it seems to come from the binary difference. “sqlite” incurred moderate 50% overhead due to many system calls for DB insertion operations, while “postmark” was 4% faster due to the fast page fault handling of large memory regions. “unpack-linux” was 23% faster due to large memory region access and relatively infrequent system calls. The Linux kernel build, a mix of CPU, memory, and disk accesses, incurred 16% overhead (16 seconds slower).

Overall, the performance of Noah was greatly affected by the frequency of system calls and page faults occurred in the applications, since the system calls are much slower and page fault handling is faster in Noah than macOS. However, as described in Section 1, the purpose of our system is not full performance compatibility but to easily run applications of other OSs at a reasonable speed. Therefore, we believe this application performance is sufficient to achieve our goal.

5.2.4 Performance Comparison on Windows

To illustrate the characteristics of OS compatibility layers, we carried out a simple performance comparison of basic system calls on Windows. We wrote a simple micro benchmark and ran it on NoahW, Cygwin, and WSL1. Cygwin is a typical user-space-only implementation of OS compatibility layers that can run some UNIX-like applications on Windows by converting UNIX system calls into Windows ones with

Table 1. System call performance comparison on NoahW, Cygwin and WSL1.

Benchmark	NoahW	Cygwin	WSL1
dup2() [calls per second]	36,723	556,453	693,309
write() (1 GiB block size) [calls per second]	0.30	0.56	0.57
fork() (global array size: 0 MiB) [ms]	106.4	219.4	2.06
fork() (global array size: 512 MiB) [ms]	338.9	789.9	32.51
fork() (global array size: 1 GiB) [ms]	458.4	1531.8	62.66

compatibility libraries. WSL1 is a recent OS compatibility layer implementation using an in-kernel subsystem that can run unmodified Linux binaries on Windows.

We measured the performance of three types of system calls: `dup2()`, `write()`, and `fork()`. `dup2()` is a simple system call that duplicates the opened file descriptor specified by the first argument and assign it to the file descriptor specified by the second argument. Since its operation inside the OS compatibility layer and kernel is simple, it can be used to measure the basic context switch cost between the user and kernel spaces. We measured the cost of issuing this system call as the number of calls per second.

The first line in Table 1 shows the results of `dup2()`. As shown in the results, the performance of NoahW was approximately 18 times slower than Cygwin. As already shown in Figure 3, the main reason of this was the slow VMM module; the cost of VM context switching was not so high. Although the low performance in primitive system calls is a primary disadvantage of our approach, we believe there is still considerable room for performance improvement by optimizing the VMM module. In addition, calling only primitive system calls frequently is uncommon in real applications, and therefore the relative cost of the calling system calls itself is smaller. In WSL1, `dup2()` was slightly faster than Cygwin. This was probably because the system call was processed in a real Linux kernel, which was highly optimized and faster than the Windows kernel. This means that the system call performance is affected by the host system that actually processes the system calls.

The next line in Table 1 shows the results of `write()`. The benchmark program repeatedly wrote a 1 GiB block of memory with the `write()` system call and measured the number of calls per second. As shown in the results, the NoahW overhead was reduced to 1.87 times. In contrast, Cygwin and WSL1 showed similar performance. This was probably because the performance of large I/O was limited by the performance of the storage device and Windows kernel. This means that the I/O system call performance is roughly determined by the host system I/O performance.

The last three lines in Table 1 shows the results of `fork()`. We measured the time to issue `fork()` and the consequent `wait4()` system calls while changing the global array size of the benchmark program to 0 MiB, 512MiB, and 1GiB

to measure the effects of the copy-on-write. As already shown in Figure 2, NoahW was much faster than Cygwin; the merit of supporting copy-on-write outperformed the demerit of slow system calls. For example, in the 1 GiB array case, NoahW was 3.3 times faster than Cygwin. On the other hand, WSL was much faster than NoahW and Cygwin. This was probably because the complicated operations of copy-on-write was processed at once in the kernel.

Overall, our approach has slow primitive system call performance although it could be improved with optimization, and better copy-on-write performance than Cygwin but much slower than WSL1. We believe this performance was good enough for casual use as described in Section 1.

6 Conclusion

In this paper, we presented the design and implementation of a novel OS compatibility layer that achieved both robustness and flexibility by exploiting virtualization technology. In our architecture, a guest process runs inside a VM, and system calls and exceptions are trapped by the monitor process with the support of the VMM module. The implementations were demonstrated to be robust because bugs in them did not lead to system crashes, and it was flexible because it could run unmodified Linux binaries and support the copy-on-write capability in Windows. The evaluation showed that Noah incurred moderate overheads on simple system calls and reasonable overheads on most other system calls, while it speeded up `exec()`, page fault handling, and large region memory access. As for macro benchmarks, the Linux kernel build time on Noah was only 16% slower than on macOS. The comparison with other OS compatibility layers had revealed the merit and demerit of our approach. Considering the almost user-space-only implementation, binary compatibility, and improved the copy-on-write performance, our approach achieved robust and flexible OS compatibility layers.

Acknowledgments

This work was partially supported by Exploratory IT Human Resources Project (MITOU Program) of Information-technology Promotion Agency, Japan (IPA) in the fiscal 2016 and JSPS KAKENHI Grant Number 16H02798. Noah is publicly available from <https://github.com/linux-noah/noah> under the MIT / GPL dual licenses.

References

- [1] Jeremy Andrus, Alexander Van't Hof, Naser AlDuaij, Christoffer Dall, Nicolas Viennot, and Jason Nieh. 2014. Cider: Native Execution of iOS Apps on Android. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2014)*. ACM, 367–382. <https://doi.org/10.1145/2541940.2541972>
- [2] Apple. 2019. Hypervisor | Apple Developer Documentation. <https://developer.apple.com/documentation/hypervisor>
- [3] Andrew Baumann, Jonathan Appavoo, Orran Krieger, and Timothy Roscoe. 2019. A Fork() in the Road. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '19)*. Association for Computing Machinery, New York, NY, USA, 14–22. <https://doi.org/10.1145/3317550.3321435>
- [4] Adam Belay, Andrea Bittau, Ali José Mashtizadeh, David Terei, David Mazières, and Christos Kozyrakis. 2012. Dune: Safe User-level Access to Privileged CPU Features. In *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2012)*. 335–348.
- [5] Hao Chen, David Wagner, and Drew Dean. 2002. Setuid Demystified. In *Proceedings of the 11th USENIX Security Symposium*. 171–190.
- [6] J. B. Chen, Y. Endo, K. Chan, D. Mazieres, A. Dias, M. Seltzer, and M. D. Smith. 1995. The Measured Performance of Personal Computer Operating Systems. In *Proceedings of the 15th ACM Symposium on Operating Systems Principles*. 299–313. <https://doi.org/10.1145/224056.224079>
- [7] Anton Chernoff, Mark Herdeg, Ray Hookway, Chris Reeve, Norman Rubin, Tony Tye, S. Bharadwaj Yadavalli, and John Yates. 1998. FX!32 A Profile-Directed Binary Translator. *IEEE Micro* 18, 2 (Mar 1998), 56–64. <https://doi.org/10.1109/40.671403>
- [8] Cygwin authors. 2019. Cygwin. <https://www.cygwin.com>
- [9] Cygwin authors. 2019. Cygwin User's Guide. <https://cygwin.com/cygwin-ug-net/cygwin-ug-net.html>
- [10] John R. Douceur, Jeremy Elson, Jon Howell, and Jacob R. Lorch. 2008. Leveraging Legacy Code to Deploy Desktop Applications on the Web. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2008)*. 339–354.
- [11] Emmanuel Dreyfus. 2001. Linux Compatibility on BSD for the PPC Platform. http://www.onlamp.com/pub/a/onlamp/2001/05/10/linux_bsd.html.
- [12] Emmanuel Dreyfus. 2002. IRIX Binary Compatibility. <http://www.onlamp.com/pub/a/bsd/2002/08/08/irix.html>.
- [13] Emmanuel Dreyfus. 2004. Mac OS X binary compatibility on NetBSD: challenges and implementation. In *Proceedings of the EuroBSDCon 2004*.
- [14] Hideki Eiraku, Yasushi Shinjo, Calton Pu, Younggyun Koh, and Kazuhiko Kato. 2009. Fast Networking with Socket-outsourcing in Hosted Virtual Machine Environments. In *Proceedings of the 2009 ACM Symposium on Applied Computing (ACM SAC 2009)*. 310–317. <https://doi.org/10.1145/1529282.1529350>
- [15] FreeBSD Documentation Project. 2004. Linux Binary Compatibility. In *The FreeBSD Handbook 3rd Edition*, Murray Stokely and Chern Lee (Eds.). Vol. 1: User Guide. Chapter 10.
- [16] J. E. Gaffney. 1984. Estimating the Number of Faults in Code. *IEEE Transactions on Software Engineering* SE-10, 4 (July 1984), 459–464. <https://doi.org/10.1109/TSE.1984.5010260>
- [17] David Golub, Randall Dean, Alessandro Forin, All Dean, Ro Forin, and Richard Rashid. 1990. Unix as an Application Program. In *Proceedings of the USENIX 1990 Summer Conference*. 87–95.
- [18] gyf304. 2016. MLton crashes and BSODs. <https://github.com/Microsoft/BashOnWindows/issues/847>.
- [19] Shinichiro Hamaji. 2018. Maloader. <https://github.com/shinh/maloader>.
- [20] Hermann Härtig, Michael Hohmuth, Jochen Liedtke, Jean Wolter, and Sebastian Schönberg. 1997. The Performance of μ -kernel-based Systems. In *Proceedings of the 16th ACM Symposium on Operating Systems Principles (SOSP 1997)*. 66–77. <https://doi.org/10.1145/268998.266660>
- [21] Paul Hohensee, Mat Myszewski, and David Reese. 1996. Wabi CPU Emulation. In *Hot Chips 8*.
- [22] Jon Howell, Bryan Parno, and John R. Douceur. 2013. Embassies: Radically Refactoring the Web. In *Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2013)*. 529–545.
- [23] Jon Howell, Bryan Parno, and John R. Douceur. 2013. How to Run POSIX Apps in a Minimal Picoprocess. In *Proceedings of the 2013 USENIX Annual Technical Conference*. 321–332.
- [24] Intel. 2017. Intel Hardware Accelerated Execution Manager (Intel HAXM). <https://software.intel.com/articles/intel-hardware-accelerated-execution-manager-intel-haxm>.
- [25] Intel. 2019. HAXM API Reference. GitHub. <https://github.com/intel/haxm/blob/master/docs/api.md>
- [26] iz0eyj. 2016. BSOD starting Vivaldi Browser. <https://github.com/Microsoft/WSL/issues/1344>.
- [27] KVM Developers. 2019. The Definitive KVM (Kernel-based Virtual Machine) API Documentation. kernel.org. <https://www.kernel.org/doc/Documentation/virtual/kvm/api.txt>
- [28] Conor Hetland Kyle C. Hale and Peter Dinda. 2017. Multiverse: Easy Conversion of Runtime Systems into OS Kernels via Automatic Hybridization. In *Proceedings of the 14th IEEE International Conference on Autonomic Computing (ICAC 2017)*.
- [29] Craig Loewen. 2019. Announcing WSL 2. Microsoft Developers Blogs. <https://devblogs.microsoft.com/commandline/announcing-wsl-2/>
- [30] Craig Loewen. 2019. A Deep Dive Into How WSL Allows Windows to Access Linux Files. Microsoft Developers Blogs. <https://devblogs.microsoft.com/commandline/a-deep-dive-into-how-wsl-allows-windows-to-access-linux-files/>
- [31] Craig Loewen. 2019. Memory Reclaim in the Windows Subsystem for Linux 2. Microsoft Developers Blogs. <https://devblogs.microsoft.com/commandline/memory-reclaim-in-the-windows-subsystem-for-linux-2/>
- [32] Larry McVoy and Carl Staelin. 1996. Imbench: Portable Tools for Performance Analysis. In *Proceedings of the USENIX 1996 Annual Technical Conference*. 279–294.
- [33] Larry McVoy and Carl Staelin. 2014. Lmbench - Tools for Performance Analysis. <https://sourceforge.net/projects/lmbench/>
- [34] Michael Stahl. 2014. Quantifying the time overhead of Cygwin make. LibreOffice mailing list. <https://lists.freedesktop.org/archives/libreoffice/2014-June/061727.html>
- [35] Microsoft. 2019. Windows Hypervisor Platform API Definitions. Microsoft Docs. <https://docs.microsoft.com/virtualization/api/hypervisor-platform/hypervisor-platform>
- [36] Microsoft. 2019. Windows Subsystem for Linux Documentation. <https://docs.microsoft.com/windows/wsl/about>
- [37] MinGW.org. 2017. MinGW | Minimalist GNU for Windows. <https://www.mingw.org>.
- [38] Phoronix Media. 2017. Phoronix Test Suite. <https://www.phoronix-test-suite.com/>.
- [39] Donald E. Porter, Silas Boyd-Wickizer, Jon Howell, Reuben Olinsky, and Galen C. Hunt. 2011. Rethinking the Library OS from the Top Down. In *Proceedings of the Sixteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XVI)*. Association for Computing Machinery, New York, NY, USA, 291–304. <https://doi.org/10.1145/1950365.1950399>
- [40] Robert ÅŮgren. 2005. Slow shell script execution on Cygwin. libtool mailing list. <https://lists.gnu.org/archive/html/libtool/2005-04/msg00068.html>

- [41] Takaya Saeki, Yuichi Nishiwaki, Takahiro Shinagawa, and Shinichi Honiden. 2017. Bash on Ubuntu on MacOS. In *Proceedings of the 8th Asia-Pacific Workshop on Systems (APSys 2017)*. Association for Computing Machinery, New York, NY, USA, Article 17, 8 pages. <https://doi.org/10.1145/3124680.3124733>
- [42] Takahiro Shinagawa. 2017. The VMX benchmark (VMXbench). <https://github.com/utshina/VMXbench>.
- [43] Jeremy Sugerman, Ganesh Venkitachalam, and Beng-Hong Lim. 2001. Virtualizing I/O Devices on VMware Workstation's Hosted Virtual Machine Monitor. In *Proceedings of the 2001 USENIX Annual Technical Conference*. 1–14.
- [44] Xiangyan Sun. 2015. Foreign LINUX - Run unmodified Linux applications inside Windows. <https://github.com/wishstudio/flinux>.
- [45] sunxiaoguang. 2017. BSOD: Probably caused by : LXCORE.SYS (LXCORE!LxpFutexEvictWaitQueue+47). <https://github.com/Microsoft/WSL/issues/2648>.
- [46] Chia-Che Tsai, Kumar Saurabh Arora, Nehal Bandi, Bhushan Jain, William Jannen, Jitin John, Harry A. Kalodner, Vrushali Kulkarni, Daniela Oliveira, and Donald E. Porter. 2014. Cooperation and Security Isolation of Library OSes for Multi-Process Applications. In *Proceedings of the Ninth European Conference on Computer Systems (EuroSys 2014)*. Association for Computing Machinery, New York, NY, USA, Article 9, 14 pages. <https://doi.org/10.1145/2592798.2592812>
- [47] Volkmar Uhlig, Joshua LeVasseur, Espen Skoglund, and Uwe Dahnowski. 2004. Towards Scalable Multiprocessor Virtual Machines. In *Proceedings of the 3rd Virtual Machine Research and Technology Symposium*. 43–56.
- [48] VMware, Inc. 2019. Adjusting Virtual Machine Memory. <https://docs.vmware.com/en/VMware-Workstation-Player-for-Windows/15.0/com.vmware.player.win.using.doc/GUID-B29FC544-B15F-4E94-882A-B383C88B5A85.html>
- [49] VMware, Inc. 2019. Overview of VMware Tools. <https://kb.vmware.com/s/article/340>
- [50] VMware, Inc. 2019. Switch to Unity View. <https://docs.vmware.com/en/VMware-Fusion/11/com.vmware.fusion.using.doc/GUID-55FD9709-1980-4D4D-9F80-2896871FD921.html>
- [51] VMware, Inc. 2019. Using Shared Folders. <https://docs.vmware.com/en/VMware-Workstation-Player-for-Windows/15.0/com.vmware.player.win.using.doc/GUID-AACE0935-4B43-43BA-A935-FC71ABA17803.html>
- [52] David A Wheeler. 2001. SLOccount.
- [53] Wine Project. 2019. WineHQ - Run Windows applications on Linux, BSD, Solaris and macOS. <https://www.winehq.org/>